EXPERIMENTAL REPRODUCIBILITY REPORT

High-rate image transfer across edge devices, Wi-Fi and Gigabit Ethernet LAN

Study: Cross-Layer Bottleneck Analysis of High Throughput Image Transfer in Edge Systems

Prepared: 20 October 2026 **Purpose:** Supervisor and manuscript-methodology review

Evidence: Device-terminal outputs, supplied sender/receiver code, original receiver/iPerf3 CSV archive, and the two master CSV datasets.

Scope of this report

This document records what can be reproduced from the available experimental evidence: devices and their roles; seven sender/receiver layouts; protocol software and behavior; measurement settings; and the checked data processing steps. Conflicts and still-unverified historical settings are documented explicitly rather than filled in with assumptions.

At a glance

Experimental design	Verified/reported configuration
Physical devices	3 Raspberry Pi 5 senders + 3 heterogeneous receivers (Jetson AGX Orin, Intel NUC, Jetson Orin Nano Super)
Network conditions	5 GHz Wi-Fi and wired Gigabit Ethernet LAN
Communication technologies	MQTT, Zenoh, Kafka, HTTP, CoAP; iPerf3 TCP as network baseline
Architecture cases	7 topologies, 1-3 senders and 1-3 receivers
Repetition plan	50 receiver-side measurement windows per network x technology x case
Original source data	156 CSV files; 8,100 device-level readings; 4,200 collective run results

Reading guide: "Confirmed by log" means a terminal output, "script-shown" means the supplied source code, and "researcher-confirmed" means the experimenter's statement. Current installations do not necessarily prove the versions in use at original collection time.

1. Evidence basis and reproducibility boundaries

The report was assembled from (a) CLI output recorded on PI1, NVIDIA Jetson AGX Orin, Intel NUC and Jetson Orin Nano; (b) sender and receiver Python scripts shared in the conversation; (c) **Raw data.zip** containing original CSV files; (d) **Master_Dataset total 8100.csv** and **Master Dataset collective 4200.csv**; and (e) the researcher's explanations of how trials were run. No new experiment was performed for this report.

A new researcher can reproduce the documented design and data transformation. Exact reconstruction of the original historical runtime still requires the final executed script snapshots, receiver-side library versions, historical network settings and full capture-size records.

Evidence labels used below

Label	Meaning
Terminal verified	Read from the current device's command output; may be newer than the original experiment.
Script shown	Explicit in provided sender/receiver script; scripts may have been edited after collection.
Researcher confirmed	Stated by the experimenter, but not independently visible in the supplied script/log.
Pending confirmation	Required for exact historical replay but not proved by current records.

2. Physical testbed and device assignments

The experiment used **three Raspberry Pi image senders** and **three heterogeneous edge receivers**. The experimenter stated the three Raspberry Pis are identical and were configured at the same time. PI1 was inspected directly; the other two devices still need an independent software snapshot if exact device-by-device version matching is required.

Device / role	CPU and RAM	Operating system and kernel	Current Ethernet
Pi1, Pi2, Pi3 **Image senders**	Raspberry Pi 5 Model B Rev 1.0; 4-core ARM Cortex-A76, up to 2.4 GHz; 8 GB each	Debian 12 Bookworm, ARM64; PI1 kernel 6.6.31+rpt-rpi-2712	PI1: 1 Gb/s, full duplex
Jetson AGX Orin **Receiver AGX**	12-core ARM CPU up to ~2.2 GHz; 64 GB RAM	Ubuntu 20.04.6 LTS, ARM64; 5.10.104-tegra; L4T R35.2.1	1 Gb/s, full duplex
Intel NUC12WSKv7 **Receiver NUC**	Intel Core i7-1270P; 12 physical cores / 16 threads; 32 GB RAM	Ubuntu 24.04.2 LTS, x86-64; 7.0.0-28-generic	1 Gb/s, full duplex
Jetson Orin Nano Super **Receiver ORIN**	6-core Cortex-A78AE up to 1.728 GHz; 8 GB RAM	Ubuntu 22.04.5 LTS, ARM64; 5.15.148-tegra; L4T R36.4.7	1 Gb/s, full duplex

Device specifications are from current CLI outputs. RAM capacities are nominal from Linux memory readings (PI1 7.9 GiB; AGX 61 GiB; NUC 30 GiB; Nano 7.4 GiB).

Device naming in the data

The original receiver folders use AGX, Nuc/NUC and Orin1/ORIN. For iPerf3, source files additionally identify Sent From Pi1, Pi2 or Pi3. These are the labels used to reconstruct per-case sender and receiver contributions.

3. Seven tested sender-receiver arrangements

All seven cases were repeated on both Wi-Fi and LAN. The topology labels in the source archive are One_to_One, One_to_Two, One_to_Three, Two_to_One, Two_to_Two, Three_to_One and Three_to_Three.

Case	Senders	Receivers	Active communication paths
1 (1-1)	1	1	Pi1 -> AGX
2 (1-2)	1	2	Pi1 -> AGX; Pi1 -> NUC
3 (1-3)	1	3	Pi1 -> AGX, NUC and ORIN
4 (2-1)	2	1	Pi1 and Pi2 -> AGX (shared receiver)
5 (2-2)	2	2	Pi1 -> AGX; Pi2 -> NUC (separate paths)
6 (3-1)	3	1	Pi1, Pi2 and Pi3 -> AGX (shared receiver)
7 (3-3)	3	3	Pi1 -> AGX; Pi2 -> NUC; Pi3 -> ORIN (parallel paths)

Case descriptions are supported by topology labels, receiver CSV folders and iPerf3 sender-target logs. They should not be confused with the number of broker instances.

Case families: Cases 1-3 increase destinations with one sender. Cases 4 and 6 place two and three senders, respectively, on a shared AGX receiver. Cases 5 and 7 use independent one-to-one paths concurrently. These differences are important when comparing single- and multiple-worker configurations.

4. Network setup

4.1 Wired LAN

PI1, AGX, NUC and Orin Nano presently negotiate **1000 Mb/s, full duplex** Ethernet links. The physical link is Gigabit Ethernet; the switch manufacturer/model, cabling category and switch configuration were not recorded in the supplied evidence. PI2 and PI3 were said to use the same setup as PI1.

4.2 Wireless network

The experiment used a **5 GHz Wi-Fi** network. The current PI1 wireless interface information lists channel **40, 5200 MHz**, with **20 MHz** channel width; the NUC current connection also reports 5200 MHz. The access point model, wireless standard, channel width on every participating device, distance to access point, interference/background load and historical channel settings were not recovered.

These readings document the observed network environment but cannot establish that every radio parameter was unchanged during original data collection. The Wi-Fi and LAN trials are therefore described as separate network conditions, not as identical physical conditions except for the medium.

4.3 Network roles and relevant ports

Service	Location / endpoint	Role
Mosquitto MQTT	PI1: 192.168.1.68:1883 in supplied Pi1 sender	MQTT broker reachable by sender/receiver
Kafka	AGX: 192.168.1.21:9092 in supplied Kafka sender	Kafka bootstrap broker endpoint
Zenoh	Worker peers: TCP port 7447	Publisher connects to peers; subscriber listens
HTTP	Receiver: TCP port 8000; /upload	JSON HTTP POST image intake
CoAP	Receiver: UDP port 5683; /office/pi1/image	CoAP PUT image intake
iPerf3	Receiver: TCP port 5201	Network-only throughput baseline

Private LAN addresses are configuration examples from supplied scripts. The Wi-Fi and LAN historical endpoints may have differed.

5. Software environment and versions

The following versions were obtained from currently installed packages on PI1 and the running broker environment. Versions found in a Python virtual environment should not be silently substituted for versions used during a historical run.

Component	Detected software / version	Evidence or qualification
PI1 system	Python 3.11.2; pip 23.0.1; system OpenCV 4.6.0	Terminal verified
MQTT environment	paho-mqtt 2.1.0; OpenCV 4.10.0.84	Project2.0/MQTT/myenv
MQTT system Python	paho-mqtt 1.6.1	Separate system installation; original interpreter unconfirmed
MQTT broker	Eclipse Mosquitto 2.0.11	PI1 system service
Zenoh environment	eclipse-zenoh 1.6.2; OpenCV 4.12.0.88	Project2.0/Zenoh/env
Kafka environment	kafka-python 2.3.0; OpenCV 4.12.0.88	Project2.0/Kafka/venv
Kafka broker	Apache Kafka 3.8.0; ZooKeeper library 3.8.4	AGX Java process / installed libs
HTTP client	requests 2.28.1 in system Python; 2.32.5 in Zenoh/env	Execution environment unknown
HTTP receiver	Python standard library http.server.HTTPServer	From receiver source code
CoAP client/server	aiocoap 0.4.17 found in Zenoh/env	Execution environment unknown
iPerf3	3.12, TCP	PI1 binary --version

Other receiver-side package versions and the exact executed Python interpreters are not yet documented.

6. MQTT broker and Kafka broker settings

6.1 Mosquitto MQTT

PI1 runs the Mosquitto system service using **/etc/mosquitto/mosquitto.conf**. The config inspection returned **listener 1883**, **persistence true**, and **allow_anonymous true**. No explicit queue size, message size or inflight-message overrides were present in the examined .conf files. The service had been running in the current snapshot; this does not prove unchanged configuration at historical collection time.

6.2 Apache Kafka and ZooKeeper

The running Kafka installation was observed as a direct Java process on AGX rather than a Docker container. It uses Kafka 3.8.0 and a local ZooKeeper service; the main configuration shows a PLAINTEXT listener and the following relevant values.

Kafka property	Main configuration	LAN/Wi-Fi copies
broker.id	0	0 / 0
listeners	PLAINTEXT://0.0.0.0:9092	PLAINTEXT://:9092
advertised.listeners	PLAINTEXT://192.168.1.21:9092	PLAINTEXT://192.168.1.135:9092
num.partitions	1	1 / 1
zookeeper.connect	localhost:2181	localhost:2181
log.dirs	/tmp/kafka-logs	Not examined in separate files
log.retention.hours	168	Not examined in separate files

Main file: ~/kafka/config/server.properties. Alternative copies exist under ~/kafka/Lan/config and ~/kafka/Wifi/config; which copies were used in the historic tests is still unconfirmed.

The producer script also sets acks="all", retries=1 and linger_ms=5; these are producer settings, not broker settings. The one-partition value refers to a Kafka broker default, not proof that every existing topic had exactly one partition.

7. Image acquisition and workload

The five communication sender scripts acquire frames from a locally attached camera through **cv2.VideoCapture(0)**, request a resolution of **640 x 480 pixels**, and encode captured frames as **JPEG, quality 95**. Each frame is encoded once; **MULTIPLY_FACTOR=10** causes ten repeated transmissions per configured topic/key/destination in the provided snippets. **REPLICAS=1** creates one topic/key per base path. Image saving at the sender is disabled with SAVE_TO_DISK=False, although counter files and logs may be written.

The source is a **live camera workload**, not a fixed image dataset. Because JPEG compression depends on image content, payload sizes were not constant and the CSVs do not retain each original JPEG byte length. One retained **test_capture.jpg** measured **43,571 bytes** and 640 x 480 pixels, last modified 9 December 2025. This is a test-file example, not proof of the exact on-wire size in all experimental runs.

7.1 Payload structure across protocols

Technology	Data submitted by sender	Image repeat/transport description
MQTT	Base64-encoded JPEG text	publish(topic, payload_b64, qos=2), 10 copies per topic
Zenoh	JPEG byte array	pub.put(jpeg_bytes), 10 copies per key
Kafka	JSON containing topic, filename, image_b64	Kafka producer.send() to configured topic; flush after each capture
HTTP	JSON containing topic, filename, image_b64	requests.post() for each configured receiver
CoAP	JPEG byte array	CoAP PUT to configured resource URI

Base64 expands a binary payload before framing, while JSON and message handling add additional overhead. These are implementation-level image pipelines rather than payload-identical raw protocol benchmarks.

7.2 Payload information needed for exact replay

To reproduce the exact image content and byte sizes, one would need retained image files or original per-frame payload-size logs, camera model and acquisition conditions. Those are not present in the supplied CSV archive. A new run could reproduce the capture and encoding procedure but not guarantee identical live camera bytes.

8. Protocol sender and receiver behavior

8.1 MQTT

Sender uses paho-mqtt, broker port 1883, base topic images/pi1, 60-second keepalive, a background network loop, QoS 2 on publish, one replica and ten publish calls for each encoded frame. The receiver subscribes to images/# and decodes received Base64, reconstructs the image with OpenCV, writes an image file, and derives throughput from files saved in its run window.

QoS verification: The researcher confirms that the intended experimental configuration used **QoS 2 for sender and receiver**. The shared receiver excerpt, however, calls **client.subscribe(topic_filter)** without an explicit qos argument; in the shown Paho API this requests QoS 0. Thus the statement "QoS 2 at both ends" is recorded as researcher-confirmed but **not independently established by the supplied receiver file**. For a reproducible artifact, the final executed receiver file or broker/subscription evidence must resolve this difference.

8.2 Zenoh

Zenoh is configured in **peer mode**. The sender calls connect/endpoints over TCP port 7447 for its configured worker peers, declares a publisher on office/pi1/image and publishes raw JPEG bytes through put(). The receiver is a peer listening at tcp/0.0.0.0:7447 and declares a subscriber for the same key. It decodes image bytes, saves them and calculates throughput from output files. The sample sender contains three worker endpoints: a topology-specific historical endpoint list still needs to be preserved for each case.

8.3 Kafka

The Kafka producer sends JSON data to images_pi1 with one topic replica in the shown code. Important producer parameters are **acks=all**, **retries=1**, **linger_ms=5** and an explicit **flush()** after the ten-copy batch. The Kafka consumer uses group_id=image-receiver-benchmark-group, auto_offset_reset=latest, enable_auto_commit=True and consumer_timeout_ms=100, with one background image processing worker.

8.4 HTTP

The HTTP sender uses Python requests to POST JSON to /upload at port 8000, with a five-second timeout, repeating an image ten times per configured destination. The receiver uses Python's standard HTTPServer and a custom BaseHTTPRequestHandler; accepted payloads are queued for image decoding and saving. The server returns HTTP 200 after queueing, rather than waiting for file-write completion. The sample script has one active HTTP destination, with optional additional destinations commented out.

8.5 CoAP

The CoAP sender uses aiocoap to transmit raw JPEG bytes as sequential PUT requests on port 5683 to /office/pi1/image. It waits for each response with a **2-second application timeout**; after **five consecutive failures**, it recreates its client context, pausing for **0.5 second**. The receiver uses aiocoap.resource and saves the received JPEG bytes directly before returning a CHANGED response. The request confirmation mode is not explicitly set in the provided sender code, so it is not labeled as a confirmed historical CoAP message-type choice.

9. Execution order, measurement window and repetitions

Experimenter account: The sender was launched and allowed to transmit continuously for approximately **2-3 hours**; receiver scripts collected their 50 sequential measurement windows, after which the sender was stopped manually. This is the provided operational description.

Script distinction: The supplied sender snippets set RUN_DURATION=10 seconds. Therefore those snippets are not sufficient on their own to reproduce the recalled 2-3-hour continuous runs. The original long-running sender versions, or the modified duration used when launched, need to be attached to a replication package. The following describes what the receiver code and preserved data demonstrate.

1. **Choose network (Wi-Fi or wired LAN), communication technology and one of Cases 1-7. Start required broker/router/server and receivers for that topology.**
2. **Start the sender processes for all active Raspberry Pis. A sender captures a frame, encodes to JPEG, and sends ten copies on the currently configured destination/topic list.**
3. **At a receiver, begin the benchmark window when the first image/message is observed. Each receiver then collects data for RUN_AFTER_FIRST_IMAGE=10 seconds.**
4. **At the end of each receiver window, record count of saved images, total saved file size and calculated throughput, delete saved images, and begin the next run. The code sets NUMBER_OF_LOOPS=50.**
5. **Retain each numbered run separately in a CSV. Where a case has multiple receivers or senders, combine device-level readings corresponding to the same network, technology, case and run number.**
6. **Use iPerf3 separately as a network throughput reference: TCP tests with -t 10, -J and server port 5201, repeated 50 times for configured targets.**

9.1 Timing and output differences that affect exact replay

Receiver timers start on their first message, while iPerf3 duration is controlled by its client test command. MQTT uses a callback-driven receiving loop. Zenoh, Kafka, HTTP and CoAP receivers implement a 60-second first-image wait. Kafka and HTTP write images via a background queue; other receivers have different processing steps. These documented details affect when an image is counted and should be retained in the code package. Neither the CSVs nor the shared excerpts prove perfect synchronization of all active receiver windows.

9.2 What is being measured

Middleware receiver CSVs compute received/saved-image application throughput as **total size of successfully saved images divided by 10 seconds**. Most receiver implementations decode and re-encode images before saving; CoAP directly writes the JPEG payload. iPerf3 reads network transfer statistics from TCP test results. These measurements are not identical to packet-level network payload throughput, and the difference should be acknowledged when interpreting protocol rankings.

Units: Although filenames and original columns say MB/s, the shown scripts divide bytes by 1024 x 1024; strictly these values represent **MiB/s**. The report retains the dataset's original column names for traceability rather than silently changing the numerical results.

10. Raw CSV archive and data dictionary

Raw data.zip contains **156 CSV files**, with 26 files for each of MQTT, Zenoh, Kafka, HTTP, CoAP and iPerf3. The folder structure separates protocol, network, receiver and/or iPerf3 sender. The 50 measurements for each condition are retained in the original CSVs.

Original receiver CSV field	Meaning
loop_number	Repeated trial index (1-50).
run_duration_seconds	Nominal receiver measurement duration, usually 10 seconds.
total_images	Number of output image files counted for the window.
total_size_mb	Sum of saved image file sizes divided by 1024^2.
images_per_second	Saved image count / nominal duration.
mb_per_second	Saved image file size (MiB) / nominal duration.
timestamp / status	Record time, and in some protocols the result status.

iPerf3 CSV files also contain target_name, target_ip, target_port, protocol, sender_transfer_mb, receiver_transfer_mb, sender_MB_per_second, receiver_MB_per_second, retransmits and status. Some iPerf3 files contain a derived COLLECTIVE_TOTAL row in addition to individual targets; this must never be counted as an additional independent device measurement.

11. Detailed reconstruction from 8,100 to 4,200

The previously provided master raw and collective CSV files were cross-checked against the original archive. Each raw-master record identifies its source CSV file, CSV row number and the original row contents. All **8,100** records map to original source records, with no extra or duplicated source rows in the master raw file.

Technology	Original device readings	Run-level aggregate rows
MQTT	1,300	700
Zenoh	1,300	700
Kafka	1,300	700
HTTP	1,300	700

Technology	Original device readings	Run-level aggregate rows
CoAP	1,300	700
iPerf3	1,600	700
Total	**8,100**	**4,200**

Each technology has 2 networks x 7 cases x 50 aggregate runs = 700; per-device readings vary by how many targets participate.

Case	Original readings across all technologies and networks	Aggregate records
1 (1-1)	600	600
2 (1-2)	1,200	600
3 (1-3)	1,800	600
4 (2-1)	700	600
5 (2-2)	1,200	600
6 (3-1)	800	600
7 (3-3)	1,800	600
Total	**8,100**	**4,200**

In shared-receiver Cases 4 and 6 the middleware CSV has one receiver reading per run, whereas the iPerf3 source files retain separate sender contributions. Thus case-level source counts differ.

11.1 Grouping procedure

For each combination of **Network, Technology, Case and Run**, the throughput contributions of distinct participating source devices are added. Do not average over the 50 runs during preparation; preserve each run as one repeated observation. For example, in **MQTT - LAN - Case 2 - Run 1**, AGX records 12.96 and NUC 21.97, giving collective throughput **34.93** in the source data's recorded MiB/s units.

For iPerf3, the retained COLLECTIVE_TOTAL source row is used where present; otherwise the individual target contributions are summed. A source's collective total must not be added again on top of individual targets. The network/case grouping yields 2 networks x 6 technologies x 7 cases x 50 run groups = **4,200**.

11.2 Audit outcome and remaining data-quality check

The 3,500 middleware aggregate throughput values equal sums of their original receiver run values; all 700 iPerf3 aggregate throughput values match the source collective totals/individual source combinations. A separate comparison also matched the 3,500 middleware values in the RQ3 preparation workbook. Some auxiliary metadata (such as component counts, transferred-image counts or durations) require cleaning in a small number of aggregate rows; this is distinct from the verified throughput column. Eight CoAP source readings are marked no_images_received and should remain identifiable rather than being silently discarded.

12. Analysis handoff and repeat-run handling

The 4,200 aggregate run records, not 4,200 independent physical setups, form the analysis dataset. Each Network x Technology x Case configuration contributes 50 consecutive observations. The paper reports repeated-measures linear mixed models using an AR(1) covariance structure over run order; the network comparison includes iPerf3, while middleware architecture and selection comparisons exclude it as appropriate.

For reproducible analysis, retain Network, Technology, Case, Run and a configuration identifier. The original JMP scripts and model settings should accompany the aggregated CSV, including fixed effects, centered-run terms, repeated subject and covariance structure. The source analysis reports include the fitted model output. This report verifies data construction only; it does not rerun or validate every mixed-model p-value.

12.1 Recommended reproducibility bundle

- Original untouched Raw data.zip archive, plus a file manifest and checksums.
- The exact final executed sender and receiver scripts for each protocol and topology, with launch commands and Python virtual environment used.
- MQTT, Kafka and Zenoh router/broker configuration snapshots used at collection time.
- A short README listing which hardware, receiver and destination addresses correspond to each case.
- Master_Dataset total 8100.csv, Master Dataset collective 4200.csv, and a deterministic aggregation script.
- JMP project/model scripts and notes on handling unsuccessful/zero-throughput windows.

13. Confirmation register: before external release

The following items must remain clearly labeled until documented. They are not reasons to remove the main experimental evidence; they separate a reconstructable study from a claim of exact byte-for-byte historical replay.

Item	Current evidence	Recommended action
MQTT receiver QoS	Researcher confirms QoS 2; supplied receiver excerpt does not pass qos on subscribe.	Locate executed receiver version showing QoS 2 or correct the reported receive-side setting.
Sender runtime	Researcher recalls 2-3 h continuous sender operation; shared scripts specify 10 s.	Archive the long-running script/config or exact duration/launch override.
Python versions used	Several versions of paho-mqtt and requests are currently installed.	Record the actual interpreter path and package freeze per historical test if recoverable.
Broker/endpoint placement	Mosquitto shown on PI1; Kafka on AGX; alternative Kafka network configs.	Confirm exact host/address and active config for each case and network.
Image size history	Live camera workload; one 43,571-byte test JPEG; no per-frame size log.	Describe variable payload rather than claiming a fixed 43,571-byte or 60-KB workload.
Network reproducibility	Current 1-Gb/s links, Wi-Fi 5 GHz channel 40 at PI1.	Add Wi-Fi AP model/standard, switch, channel width and experiment-time settings when available.
Receiver side software	Receiver hardware known; runtime dependency versions not yet inventoried.	Capture receiver package versions and storage/media where outputs were written.

Item	Current evidence	Recommended action
Run synchronization	Run number matched across devices in the master; absolute synchronization not demonstrated.	Preserve orchestration/timestamps or clarify the run pairing convention.

No unverified value should be presented as a directly measured experimental fact.

14. Reference command and file inventory

14.1 Useful device commands

```
cat /proc/device-tree/model
lscpu
free -h
cat /etc/os-release
uname -r
ip -br link
sudo ethtool <ethernet-interface>
```

14.2 Software inventory commands

```
python3 --version
python3 -m pip list --format=freeze
iperf3 --version
mosquitto -h
ps -ef | grep -Ei "[k]afka|[z]ookeeper"
grep -E "^(broker.id|listeners|advertised.listeners|num.partitions|zookeeper.connect)=" ~/kafka/config/server.properties
```

14.3 Data audit file names

- Raw data.zip - 156 original CSV files grouped by protocol, network and participating device.
- Master_Dataset total 8100.csv - traceable device-level master with source file, row and original JSON field.
- Master Dataset collective 4200.csv - one collective total per network, technology, case and run.
- RQ3_Collective_JMP_Preparation(1).xlsx - corresponding middleware analysis preparation table.
- Sender and receiver Python scripts - supplied in the research conversation; preserve unchanged copies with version hashes.

Report conclusion. The study's physical participants, seven experimental arrangements, network and protocol implementation approach, nominal measurement design, and CSV aggregation have been documented in detail. The source-to-master throughput transformation has been verified. Full historical reproducibility requires the outstanding confirmations listed in Section 13, most importantly the exact executed sender/receiver scripts and the MQTT subscription QoS mismatch.